

Class3

test: 约瑟夫环问题

```
#include <stdio.h>
#include <stdlib.h>

// 定义结构体，表示双向循环链表的节点
typedef struct Node {
    int val; // 存储编号
    struct Node* prev; // 指向前一个节点的指针
    struct Node* next; // 指向后一个节点的指针
} Node;

int main() {
    int n, k;
    scanf("%d %d", &n, &k);

    // 构造双向循环链表
    Node* head = (Node*)malloc(sizeof(Node)); // 头结点
    head->val = 1;
    head->prev = NULL;
    head->next = NULL;

    // 尾结点，初始时和头结点一样
    // Node* tail = /*-----填空1-----*/;
    Node* tail = head;

    for (int i = 2; i <= n; i++) {

        // 创建新的节点
        // Node* node = /*-----填空2-----*/;
        Node* node = (Node*)malloc(sizeof(Node));
        node->val = i ;

        // tail->next = /*-----填空3-----*/; // 把新节点接在尾部
        tail->next = node;

        node->prev = tail;

        // 新节点的后继是头结点
        // node->next = /*-----填空4-----*/;
        node->next = head;

        // 头结点的前驱是新节点
        // head->prev = /*-----填空5-----*/;
        head->prev = node;
    }
}
```

```

        // tail = /*-----填空6-----*/; // 更新尾结点
        tail = node; // 更新尾结点
    }

    // 模拟出圈过程
    Node* cur = head; // 从头结点开始数数
    while (n--) { // 循环 n 次, 直到所有人都出圈
        for (int i = 1; i < k; i++) {
            cur = cur->next; // 数到第 k 个人之前, 每次移动一个节点
        }
        printf("%d ", cur->val); // 输出当前节点的编号

        // 把当前节点cur从链表中删除

        /*-----填空7-----*/
        cur->prev->next = cur->next;
        cur->next->prev = cur->prev;
        /*-----填空7-----*/

        Node* tmp = cur->next; // 把当前节点保存起来
        free(cur); // 释放当前节点的内存
        cur = tmp; // 把指针移到下一个节点, 从下一个节点开始数数
    }

    return 0;
}

```

KMP

前缀函数 $\pi(i)$

合并串-计算前缀函数

时间复杂度高

简化

二叉树

节点链表

```

struct Node{
    int val;
    Node *left,*right;
}

```

数组链表: 优点-可以用数学算

双链数组：维护两个数组

```
Left[n];  
Right[n];
```

DFS与BFS：二叉树的最大深度

- DFS: $\max(\text{left}, \text{right}) + 1$;
- BFS: 用队列来维护等待被访问的结点: `queue que[n]`

挨个加入left和right, 存入队列

挨个访问`que[i]`

堆

堆是一个**特定的**完全二叉树

- 大顶堆:

任意节点值 大于 左右子节点的值

- 小顶堆:

任意节点值 小于 左右子节点的值

入堆 - 怎么保证仍然是一个完全二叉树、性质正确

```
step1: 放入特定的位置  
step2: 比较大小, 与root交换位置  
step3: 重复循环
```

出堆 - 怎么保证不是两个子树?

```
step1: 堆顶和堆底互换  
step2: 删除堆底  
step2: (大顶堆) 堆顶选择和左右最大的交换
```

最值更新：优先队列

问题：返回n个元素的前k大元素

二分查找：二叉搜索树

规定：左边比右边小
要求：严格有序
利用中序遍历：天然得到有序队列

删除节点

平衡二叉树

哈夫曼编码

权重：节点本身的值
值：离根的距离
算法：贪心

合并与查找：无序归类

问题：
使用“M a b”，合并 a, b 的分类
使用“Q a b”，查询 a, b 是否同样分类

思路
建立二叉树：合并分类
查找二叉树：看是否为一个根节点

更高级的树

- AVL 树
- 红黑树
- 线段树